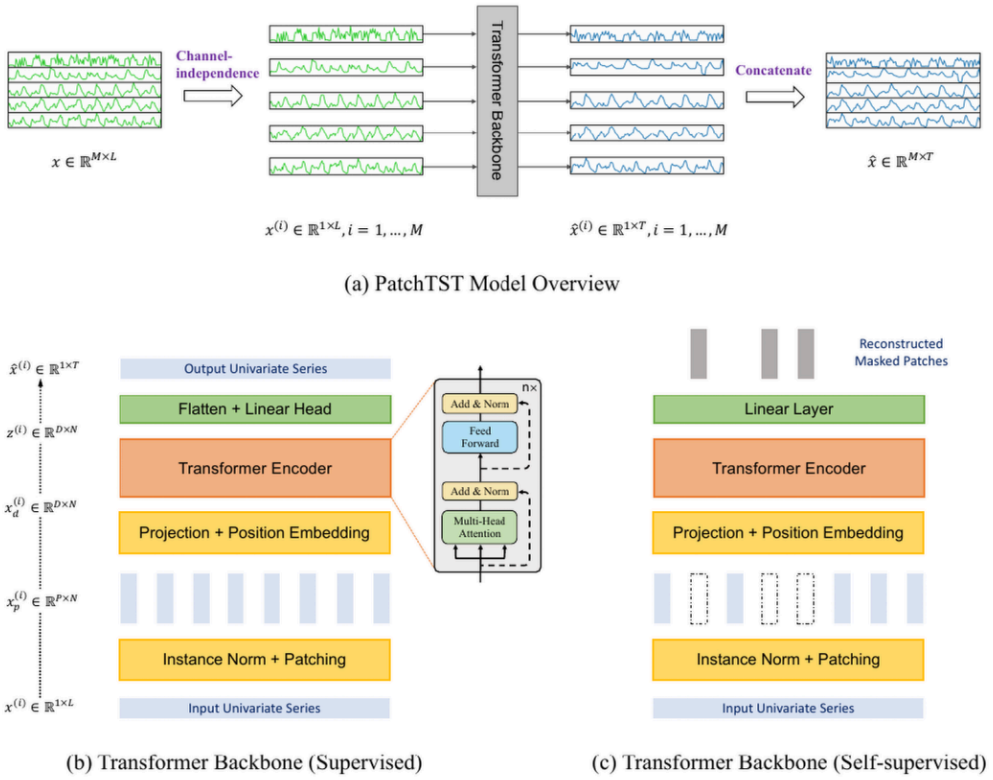
引言

时间序列预测在许多实际应用中具有重要意义，如能源预测、交通流量预测、气象预测等。随着深度学习技术的进步，基于Transformer的模型在时间序列预测中表现出了强大的能力。然而，传统Transformer模型存在计算复杂度高、内存消耗大等问题，尤其在长序列预测中效果不佳。**PatchTST**模型通过创新性的设计，解决了这些问题，提出了**分块**（Patching）和**通道独立性**（Channel-independence）的解决方案，显著提高了时间序列预测的效率和精度。

模型



> **Tip**
> **表示学习（Representation Learning）**，也常被称为特征学习（Feature Learning），是指让模型自动地从原始数据中发现和提取“特征”或“表示”的技术，而不是依靠人工手动设计规则。
> 可以暂且认为，表示学习可分为以下三种：
>
> - **有监督表示学习**：在学习特征的同时，利用标签（如未来的真实值）来纠正模型，让特征直接服务于特定目标。
> - **自监督表示学习**：不利用标签，通过数据自身的结构（如掩码 Patch）来学习特征。这是本文的核心亮点。
> - **迁移学习（Transfer Learning）**：在数据集 A 上学到的表示能力，直接应用到数据集 B 上。

有监督学习

Channel Independent

作者强调，每个时间序列都将拥有自己的潜在表示，这些表示通过**共享权重机制**进行交叉学习。这种设计允许预训练数据包含与下游数据不同数量的时间序列，这在其他方法中可能是不可行的。

> - 这种机制直接把channel维度和batch合并，也就是`[B, T, C] -> [B * N, T, 1] -> [B * N, T, d_model]`，这是后面被广泛遵循的处理方式。
> - 正是这样的设置，统一了模型处理的数据的第三维的形状，从而能够实现上文所说预训练和下游数据可以有"不同数量"（也就是**原始通道维范围**可以不同）
> - "潜在表示"：指映射到`d_model`维度上；
> - "共享权重"：指几个通道合并到`Batch`维度一起送进模型，从而可以说共享模型的权重。

Instance Norm

每个子序列块会先经过标准化处理，使其均值为0，标准差为1，从而减少不同序列之间的分布差异，避免训练过程中可能出现的梯度消失问题；输出预测的时候我们再加回均值和偏差。

$x_i' = \frac{x_i - \mu(x_i)}{\sigma(x_i)}$

其中 $\mu(x_i)$ 和 $\sigma(x_i)$分别是子序列块 x_i 的均值和标准差。

Patching Design

在PatchTST中，时间序列首先被切分成多个**子序列块（patches）**。每个子序列块代表时间序列中的一段连续区间，而不是直接将单个时间步作为输入。假设每个时间序列的长度为 L，我们将其切分为长度为 P 的子序列，并根据设定的**步长（stride）** S 决定每个块之间的重叠程度，得到 $N = \left\lfloor \frac{L-P}{S} \right\rfloor + 2$个子序列块。

> **Note**
> 这里在分块前，向原始序列末尾填充了 S 个重复的末位值 $x_L^{(i)} \in \mathbb{R}$。

这样分块相比传统的Transformer，有以下优势：

- **保留局部语义**：单个时间步缺乏语义意义，而通过聚合时间步，可以捕捉更全面的局部语义信息。
- **降低计算复杂度**：将输入序列长度L 降低为 $N \approx L/S$（S 为步长），使得注意力图的计算和内存使用呈二次方级减少。
- **支持更长历史**：由于计算量的降低，模型可以处理更长的Look-back window，从而提取更多信息。

Transformer Encoder

我们使用原生的 Transformer 编码器将观测信号映射到潜在表示（latent representations）。每个分块后的时间序列子序列会经过以下步骤进行嵌入：

1. **实例标准化（Instance Normalization）**：前面已经说过详细方法；
2. **编码**：
   - 每个子序列块先经过一个线性投影$W_p \in \mathbb{R}^{D \times P}$ 被映射到维度为 D 的 Transformer 潜在空间；

- 添加可学习的**位置编码**$W_{pos} \in \mathbb{R}^{D \times N}$，确保模型能够识别时间序列中各个时间步的位置；
- $x_d^{(i)} = W_p x_p^{(i)} + W_{pos}$，其中 $x_d^{(i)} \in \mathbb{R}^{D \times N}$ 表示将输入到 Transformer 编码器的数据。
- 多头注意力中的每个头 $h = 1, \ldots, H$ 将其转换为查询矩阵 $Q_h^{(i)} = (x_d^{(i)})^T W_h^Q$、键矩阵 $K_h^{(i)} = (x_d^{(i)})^T W_h^K$ 和值矩阵 $V_h^{(i)} = (x_d^{(i)})^T W_h^V$；
- 使用缩放点积来获取注意力输出 $O_h^{(i)} \in \mathbb{R}^{D \times N}$ $(O_h^{(i)})^T = Attention(Q_h^{(i)}, K_h^{(i)}, V_h^{(i)}) = Softmax\left(\frac{Q_h^{(i)}(K_h^{(i)})^T}{\sqrt{d_k}}\right) V_h^{(i)}$
- 其他设定如展平层等可以看结构图，和传统Transformer类似。

1. Flatten 层：将特征“拍扁”

Transformer 编码器的输出 $z^{(i)}$ 是一个二维矩阵，其维度是 $D \times N$（D 是隐藏层维度，N 是分块后的块数量）。

- **操作**：它将这个矩阵展平成一个一维的长向量，长度为 $D \times N$。
- **目的**：为了能够输入到接下来的全连接线性层中。

2. Linear Head（线性头）：生成预测值

这实际上是一个全连接层（Linear Layer）。

- **映射**：它接收展平后的长向量，通过矩阵运算将其映射到目标长度 T（即你想预测的未来时间跨度）。
- **结果**：最终输出预测序列 $\hat{x}^{(i)} \in \mathbb{R}^{1 \times T}$。

自监督学习

> - 经典的时序模型都是采用Supervised Learning范式来进行预测，也就是未来的真实数据其实就是标签，模型将预测的未来数据和真实值对比，计算误差（如MSE）从而确定优化方向。
> - 而这节的大意是，`PatchTST`可以先进行一种自监督**预训练**，然后一些迁移到小数据量的任务上来fine tuning（通常这个阶段也被称为所谓**“下游”**），效果也很好，这是由它的design决定的。
> - 所谓**迁移学习**，就是**预训练+在下游其他任务上微调**。

自监督学习

- 自监督学习是一种机器学习范式。它的核心思想是：**从数据本身挖掘“标签”来作为监督信号**。
- 有监督学习需要人类手动打标签，比如给出序列的真实值；
- 自监督学习通过mask autoencoder来实现：随机隐藏一部分输入序列的数据，让模型预测被隐藏的部分。

为什么模型也适合自监督？

- 本模型实现自监督学习，就是通过随机mask掉一部分patch（设置为0），然后根据**MSE Loss**（MSE是本模型指定的Loss Metric）训练，来重建这些patch。
- 如果不用patching，则每次mask只是以时间点为单位，这是有弊端的，具体如下：
  - 这些时间点可以通过前后的值轻松地插值补全，而不需要学习全局特征，从而可能无法对整体序列有什么理解。
  - 计算效率低；

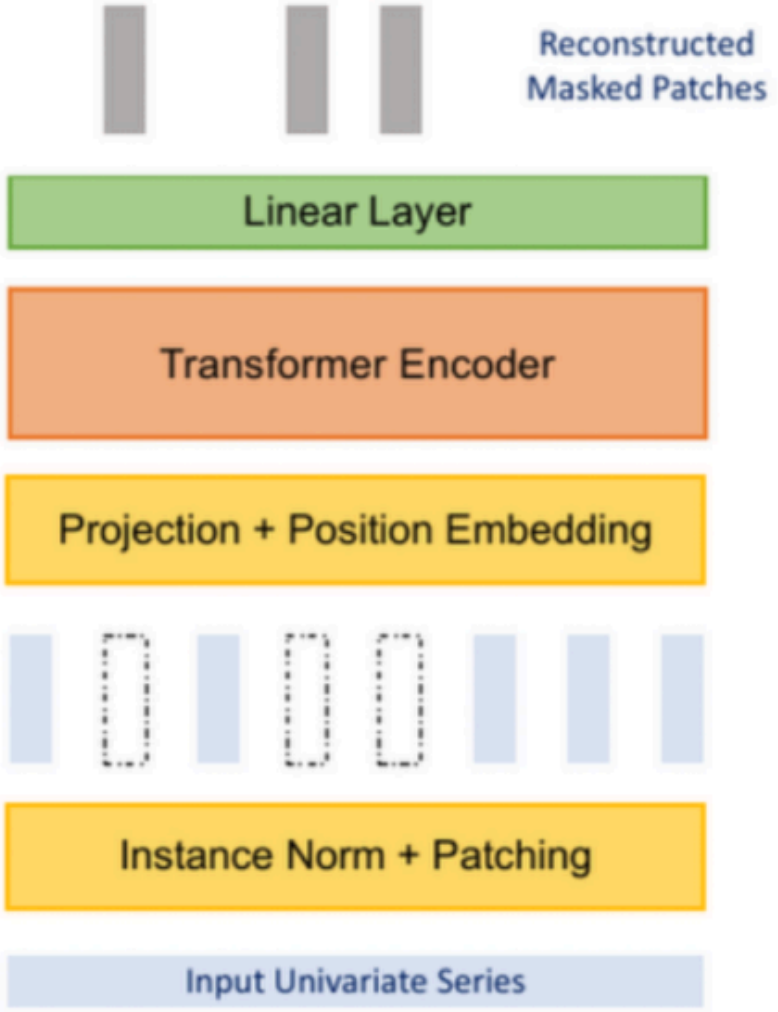
- 用于预测任务的输出层设计可能会很麻烦 。如果预测头需要处理过大的参数矩阵，可能会在下游训练样本稀缺时导致过拟合 。

> **Tip**
>
> **1. 为什么“参数矩阵”会过大?**
>
> 在传统的点对点（Point-wise）Transformer 模型中（如论文提到的 TST 模型 ），每个时间步都是一个输入标记（Token）。
>
> - **维度的累积**：假设你的输入序列长度为 L，每个时间步的隐藏表示维度为 D 。那么，整个序列的特征表示就是一个大小为 $L \times D$ 的矩阵 。
> - **输出的要求**：如果你需要预测 M 个变量在未来 T 个时间步的值，你的输出目标大小就是 $M \times T$ 。
> - **恐怖的线性层**：为了从输入映射到输出，你需要一个权重矩阵 W。在传统设计中，这个矩阵的维度是 $(L \cdot D) \times (M \cdot T)$ 。
>
> **举例**：如果 $L = 512, D = 128, M = 21, T = 96$，这个矩阵的参数量将达到数亿级别。如果其中任何一个数值增加，参数量会呈爆炸式增长 。

参数大，所以就会学习到太详细的噪声，从而导致过拟合。原文叙述如下：

Second, the design of the output layer for forecasting task can be troublesome. Given the representation vectors $\boldsymbol{z}_t \in \mathbb{R}^D$ corresponding to all L time steps, mapping these vectors to the output containing M variables each with prediction horizon T via a linear map requires a parameter matrix $\mathbf{W}$ of dimension $(L \cdot D) \times (M \cdot T)$. This matrix can be particularly oversized if either one or all of these four values are large. This may cause overfitting when the number of downstream training samples is scarce.

自监督训练的架构改动



(c) Transformer Backbone (Self-supervised)

- 预测头被移除，变成一个 $D \times P$ 的线性层，通过这个先行层；
- 每个输入序列必须被划分为规则的**非重叠块（patch）**，否则被mask掉的块的信息会在已知块中泄露。
- *mask的流程是在编码之前做的，是对mask之后的原始序列编码，而被mask块的位置编码仍然会被保留，这也是模型能够还原被mask块的关键！*

自监督方式预测的实验流程

1. 自监督训练100轮，获得预训练模型；
2. 在下游进行有监督训练，有两种训练方法：
   - Linear Probing：只训练模型的**线性预测头**，其他部分冻结，训练20轮；
   - End-to-end Fine Tuning：先训练10轮的**线性预测头**，其他部分冻结（预热）；然后解冻网络其他部分，整体fine-tuning，这一阶段进行20轮。

> 论文的实验证明第二种方法比单纯的linear probing效果好。

自监督方法对比实验

- 自监督学习有两个大的派系：
  - **生成/重构式自监督（Generative/Reconstructive SSL）**：代表作是 **Masked Autoencoders (MAE)**。PatchTST 采用的就是这种方法——遮住一部分数据，让模型“重构”出来。
  - **对比式自监督（Contrastive SSL）**：通过让模型区分“相似”和“不相似”的样本来学习特征。论文中提到的 **BTSF**、**TS2Vec**、**TNC**、**TS-TCC** 都是这种“对比学习派系”的代表。

    对比学习（Contrastive Learning） 实际上是自监督学习的一个**子集**，它的主要思想是让模型自己学习区分出**正样本**（相似的样本）和**负样本**（不相似的样本），从而获取数据的高阶特征表示；它要求模型在特征空间中能够**缩短**正样本对之间的距离（Pull），而**拉大**负样本对之间的距离（Push）。

> **Important**
> 如果有时间，应该系统了解一下对比学习，读一下这里提到的几种对比式自监督方法。

- 作者列出一些时序中较为先进的**对比学习表示方法**，然后将上述这个自监督方法和以上这些对比学习式自监督方法进行效果比较，证明了本文提出的路径更有效。

实验数据

有监督学习下与其他时序模型的对比

Models		PatchTST/64		PatchTST/42		DLinear		FEDformer		Autoformer		Informer		Pyraformer		LogTrans	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	**0.149**	**0.198**	0.152	0.199	0.176	0.237	0.238	0.314	0.249	0.329	0.354	0.405	0.896	0.556	0.458	0.490
	192	**0.194**	**0.241**	0.197	0.243	0.220	0.282	0.275	0.329	0.325	0.370	0.419	0.434	0.622	0.624	0.658	0.589
	336	**0.245**	**0.282**	0.249	0.283	0.265	0.319	0.339	0.377	0.351	0.391	0.583	0.543	0.739	0.753	0.797	0.652
	720	**0.314**	**0.334**	0.320	0.335	0.323	0.362	0.389	0.409	0.415	0.426	0.916	0.705	1.004	0.934	0.869	0.675
Traffic	96	**0.360**	**0.249**	0.367	0.251	0.410	0.282	0.576	0.359	0.597	0.371	0.733	0.410	2.085	0.468	0.684	0.384
	192	**0.379**	**0.256**	0.385	0.259	0.423	0.287	0.610	0.380	0.607	0.382	0.777	0.435	0.867	0.467	0.685	0.390
	336	**0.392**	**0.264**	0.398	0.265	0.436	0.296	0.608	0.375	0.623	0.387	0.776	0.434	0.869	0.469	0.734	0.408
	720	**0.432**	**0.286**	0.434	0.287	0.466	0.315	0.621	0.375	0.639	0.395	0.827	0.466	0.881	0.473	0.717	0.396
Electricity	96	**0.129**	**0.222**	0.130	**0.222**	0.140	0.237	0.186	0.302	0.196	0.313	0.304	0.393	0.386	0.449	0.258	0.357
	192	**0.147**	**0.240**	0.148	**0.240**	0.153	0.249	0.197	0.311	0.211	0.324	0.327	0.417	0.386	0.443	0.266	0.368
	336	**0.163**	**0.259**	0.167	0.261	0.169	0.267	0.213	0.328	0.214	0.327	0.333	0.422	0.378	0.443	0.280	0.380
	720	**0.197**	**0.290**	0.202	0.291	0.203	0.301	0.233	0.344	0.236	0.342	0.351	0.427	0.376	0.445	0.283	0.376
ILI	24	**1.319**	**0.754**	1.522	0.814	2.215	1.081	2.624	1.095	2.906	1.182	4.657	1.449	1.420	2.012	4.480	1.444
	36	1.579	0.870	**1.430**	**0.834**	1.963	0.963	2.516	1.021	2.585	1.038	4.650	1.463	7.394	2.031	4.799	1.467
	48	**1.553**	**0.815**	1.673	0.854	2.130	1.024	2.505	1.041	3.024	1.145	5.004	1.542	7.551	2.057	4.800	1.468
	60	**1.470**	**0.788**	1.529	0.862	2.368	1.096	2.742	1.122	2.761	1.114	5.071	1.543	7.662	2.100	5.278	1.560
ETTh1	96	**0.370**	0.400	0.375	**0.399**	0.375	**0.399**	0.376	0.415	0.435	0.446	0.941	0.769	0.664	0.612	0.878	0.740
	192	0.413	0.429	0.414	0.421	**0.405**	**0.416**	0.423	0.446	0.456	0.457	1.007	0.786	0.790	0.681	1.037	0.824
	336	**0.422**	0.440	0.431	**0.436**	0.439	0.443	0.444	0.462	0.486	0.487	1.038	0.784	0.891	0.738	1.238	0.932
	720	**0.447**	0.468	0.449	**0.466**	0.472	0.490	0.469	0.492	0.515	0.517	1.144	0.857	0.963	0.782	1.135	0.852
ETTh2	96	**0.274**	0.337	**0.274**	**0.336**	0.289	0.353	0.332	0.374	0.332	0.368	1.549	0.952	0.645	0.597	2.116	1.197
	192	0.341	0.382	**0.339**	**0.379**	0.383	0.418	0.407	0.446	0.426	0.434	3.792	1.542	0.788	0.683	4.315	1.635
	336	**0.329**	0.384	0.331	**0.380**	0.448	0.465	0.400	0.447	0.477	0.479	4.215	1.642	0.907	0.747	1.124	1.604
	720	**0.379**	**0.422**	**0.379**	**0.422**	0.605	0.551	0.412	0.469	0.453	0.490	3.656	1.619	0.963	0.783	3.188	1.540
ETTm1	96	0.293	0.346	**0.290**	**0.342**	0.299	0.343	0.326	0.390	0.510	0.492	0.626	0.560	0.543	0.510	0.600	0.546
	192	0.333	0.370	**0.332**	0.369	0.335	**0.365**	0.365	0.415	0.514	0.495	0.725	0.619	0.557	0.537	0.837	0.700
	336	0.369	0.392	**0.366**	0.392	0.369	**0.386**	0.392	0.425	0.510	0.492	1.005	0.741	0.754	0.655	1.124	0.832
	720	**0.416**	**0.420**	0.420	0.424	0.425	0.421	0.446	0.458	0.527	0.493	1.133	0.845	0.908	0.724	1.153	0.820
ETTm2	96	0.166	0.256	**0.165**	**0.255**	0.167	0.260	0.180	0.271	0.205	0.293	0.355	0.462	0.435	0.507	0.768	0.642
	192	0.223	0.296	**0.220**	**0.292**	0.224	0.303	0.252	0.318	0.278	0.336	0.595	0.586	0.730	0.673	0.989	0.757
	336	**0.274**	**0.329**	0.278	**0.329**	0.281	0.342	0.324	0.364	0.343	0.379	1.270	0.871	1.201	0.845	1.334	0.872
	720	**0.362**	**0.385**	0.367	**0.385**	0.397	0.421	0.410	0.420	0.414	0.419	3.001	1.267	3.625	1.451	3.048	1.328

Table 3: Multivariate long-term forecasting results with supervised PatchTST. We use prediction lengths $T \in \{24, 36, 48, 60\}$ for ILI dataset and $T \in \{96, 192, 336, 720\}$ for the others. The best results are in **bold** and the second best are underlined.

> **模型变体 (Model Variants)**。 我们在表 3 中提出了 PatchTST 的两个版本 。PatchTST/64 意味着输入块的数量为 64，使用了 $L = 512$ 的回溯窗口 。PatchTST/42 意味着输入块的数量为 42，使用了默认的 $L = 336$ 回溯窗口 。两者都使用块长度 $P = 16$ 和步长 $S = 8$ 。因此，我们可以使用 PatchTST/42 作为与 DLinear 和其他基于 Transformer 模型的公平比较，并使用 PatchTST/64 在更大的数据集上探索更好的结果 。更多实验细节在附录 A.1 中提供 。

自监督学习下与其他时序模型的对比

Models		PatchTST						DLinear		FEDformer		Autoformer		Informer	
		Fine-tuning		Lin. Prob.		Sup.									
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	**0.144**	**0.193**	0.158	0.209	0.152	0.199	0.176	0.237	0.238	0.314	0.249	0.329	0.354	0.405
	192	**0.190**	**0.236**	0.203	0.249	0.197	0.243	0.220	0.282	0.275	0.329	0.325	0.370	0.419	0.434
	336	**0.244**	**0.280**	0.251	0.285	0.249	0.283	0.265	0.319	0.339	0.377	0.351	0.391	0.583	0.543
	720	**0.320**	**0.335**	0.321	0.336	**0.320**	**0.335**	0.323	0.362	0.389	0.409	0.415	0.426	0.916	0.705
Traffic	96	**0.352**	**0.244**	0.399	0.294	0.367	0.251	0.410	0.282	0.576	0.359	0.597	0.371	0.733	0.410
	192	**0.371**	**0.253**	0.412	0.298	0.385	0.259	0.423	0.287	0.610	0.380	0.607	0.382	0.777	0.435
	336	**0.381**	**0.257**	0.425	0.306	0.398	0.265	0.436	0.296	0.608	0.375	0.623	0.387	0.776	0.434
	720	**0.425**	**0.282**	0.460	0.323	0.434	0.287	0.466	0.315	0.621	0.375	0.639	0.395	0.827	0.466
Electricity	96	**0.126**	**0.221**	0.138	0.237	0.130	**0.222**	0.140	0.237	0.186	0.302	0.196	0.313	0.304	0.393
	192	**0.145**	**0.238**	0.156	0.252	0.148	0.240	0.153	0.249	0.197	0.311	0.211	0.324	0.327	0.417
	336	**0.164**	**0.256**	0.170	0.265	0.167	0.261	0.169	0.267	0.213	0.328	0.214	0.327	0.333	0.422
	720	**0.193**	**0.291**	0.208	0.297	0.202	**0.291**	0.203	0.301	0.233	0.344	0.236	0.342	0.351	0.427

Table 4: Multivariate long-term forecasting results with self-supervised PatchTST. We use prediction lengths $T \in \{96, 192, 336, 720\}$. The best results are in **bold** and the second best are underlined.

> 这里都用了42个输入块的数据做对比，也就是 `seq_len = 336`。

迁移学习有效性

Models		PatchTST						DLinear		FEDformer		Autoformer		Informer	
		Fine-tuning		Lin. Prob.		Sup.									
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	**0.145**	**0.195**	0.163	0.216	0.152	0.199	0.176	0.237	0.238	0.314	0.249	0.329	0.354	0.405
	192	**0.193**	**0.243**	0.205	0.252	0.197	**0.243**	0.220	0.282	0.275	0.329	0.325	0.370	0.419	0.434
	336	**0.244**	**0.280**	0.253	0.289	0.249	0.283	0.265	0.319	0.339	0.377	0.351	0.391	0.583	0.543
	720	0.321	0.337	**0.320**	0.336	**0.320**	**0.335**	0.323	0.362	0.389	0.409	0.415	0.426	0.916	0.705
Traffic	96	0.388	0.273	0.400	0.288	**0.367**	**0.251**	0.410	0.282	0.576	0.359	0.597	0.371	0.733	0.410
	192	0.400	0.277	0.412	0.293	**0.385**	**0.259**	0.423	0.287	0.610	0.380	0.607	0.382	0.777	0.435
	336	0.408	0.280	0.425	0.307	**0.398**	**0.265**	0.436	0.296	0.608	0.375	0.623	0.387	0.776	0.434
	720	0.447	0.310	0.457	0.317	**0.434**	**0.287**	0.466	0.315	0.621	0.375	0.639	0.395	0.827	0.466

Table 5: Transfer learning task: PatchTST is pre-trained on Electricity dataset and the model is transferred to other datasets. The best results are in **bold** and the second best are underlined.

表示学习方法对比

Models		IMP.	PatchTST				BTSF		TS2Vec		TNC		TS-TCC	
			Transferred		Self-supervised									
Metrics		MSE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	24	42.3%	**0.312**	**0.362**	0.322	0.369	0.541	0.519	0.599	0.534	0.632	0.596	0.653	0.610
	48	44.7%	**0.339**	**0.378**	0.354	0.385	0.613	0.524	0.629	0.555	0.705	0.688	0.720	0.693
	168	34.5%	0.424	0.437	**0.419**	**0.424**	0.640	0.532	0.755	0.636	1.097	0.993	1.129	1.044
	336	48.5%	0.472	0.472	**0.445**	**0.446**	0.864	0.689	0.907	0.717	1.454	0.919	1.492	1.076
	720	48.8%	0.508	0.507	**0.487**	**0.478**	0.993	0.712	1.048	0.790	1.604	1.118	1.603	1.206

Table 6: Representation learning methods comparison. Column name *transferred* implies pre-training PatchTST on Traffic dataset and transferring the representation to ETTh1, while *self-supervised* implies both pre-training and linear probing on ETTh1. The best and second best results are in **bold** and underlined. IMP. denotes the improvement on best results of PatchTST compared to that of baselines, which is in the range of 34.5% to 48.8% on various prediction lengths.

总结

- 本文引入两个核心组件：**分块（patching）** 和 **通道独立（channel-independent）** 结构，提出了一种针对时间序列预测任务的有效 Transformer 模型设计。
- 与以往的工作相比，该模型能够**捕捉局部语义信息**，并从**更长的回溯窗口**中获益。模型不仅**在有监督学习中优于其他基准模型**，在**自监督表示学习**和**迁移学习**方面也具有足够的潜力。
- 模型展现出成为未来 Transformer 预测工作基础模型的潜力，并可作为时间序列基础模型的一个构建块。分块设计虽然简单，但已被证明是一个有效的算子，可以轻松迁移到其他模型中。
- 另一方面，通道独立设计可以被进一步挖掘，以整合不同通道之间的相关性。妥善建模跨通道依赖性将是未来的一个重要步骤。

> **Note**
> 模型关于通道独立的归纳偏置比较武断，所以就像论文总结所说的一样，探寻通道之间的依赖性也是很重要的事情怎么定义相关性，怎么发掘相关性，怎么利用相关性处理，都是问题。